

PriceRadar

AI-Powered Multi-Platform Price Tracker

Full Technical Reference & Development Guide

Field	Value
Project Name	PriceRadar
Type	Portfolio Project — Full Stack + ML + DevOps
Developer	Preksha Upadhyay
Institution	IIIT Surat — B.Tech ECE 2022-2026
Current Status	Phase 1 Complete (Database)
Target Deadline	8 Days (Weekend Build)
Document Date	May 2026

■ This document is the single source of truth for the PriceRadar project. It contains goals, architecture, tech stack decisions, folder structure, development phases, constraints, and AI prompting context. Use it when starting new AI conversations so the assistant has full context.

Table of Contents

- Section 1** — Project Overview & Goals
- Section 2** — Constraints & Honest Limitations
- Section 3** — Final Tech Stack (With Reasons)
- Section 4** — System Architecture
- Section 5** — Database Design
- Section 6** — Service-by-Service Breakdown
- Section 7** — ML Pipeline
- Section 8** — DevOps & Deployment
- Section 9** — Folder Structure
- Section 10** — Development Phases & Current Status
- Section 11** — Environment Variables Reference
- Section 12** — API Endpoints Reference
- Section 13** — Interview Talking Points
- Section 14** — AI Prompting Context

What Is PriceRadar?

PriceRadar is a web application where users search for products on Amazon India, set a target price, and receive email notifications when the price drops below their threshold. The system automatically scrapes prices once per day, runs three ML models to analyse deal quality, and stores the full price history for charting.

The project is intentionally scoped as a portfolio piece for a new graduate job search. Every technical decision was made to maximise interview talking points while keeping the entire stack free to build and host.

Primary Goal

Produce a live, deployed, ML-powered full-stack application that demonstrates backend engineering, system design, DevOps, and machine learning skills in a single coherent project — something an interviewer can open on a laptop and explore.

Secondary Goals

- Cover every skill listed on the resume in one project
- Generate genuine system design discussion material for interviews
- Deploy to production with CI/CD — not just a localhost demo
- Be completely free to build, host, and run indefinitely
- Complete within one focused weekend

What the User Can Do

- Register and log in securely
- Search for any Amazon India product by name
- View current price across Amazon
- See a 30-day price history chart
- Set a target price alert for any product
- Receive an email when price drops below their target
- View ML deal score, price prediction, and fake discount warning
- See all past alerts in their notification history
- View analytics — savings summary and price trends

What Makes This Impressive as a Portfolio Project

- Live production deployment with CI/CD pipeline — not just localhost
- Three ML models running in production, not just in a notebook
- Real system design patterns: job queue, pub/sub, circuit breaker, caching, rate limiting
- Multi-service architecture with independent deployments
- Fake discount detector — unique differentiator no existing tool does well
- Every component has a real interview talking point attached to it

SECTION 2 — Constraints & Honest Limitations

Hard Constraints

■ These are non-negotiable constraints decided during planning. Every architectural decision must respect them.

Constraint	Decision	Reason
API Budget	100 calls/month on RapidAPI free tier	Cannot afford paid tier. Budget = ~3 calls/day.
Platforms	Amazon India only for now	SerpAPI not configured yet. Add later.
Scrape Frequency	Once per day maximum	Conserve API quota. 100 calls / 30 days = 3 products max.
Mock/Fake Data	NO mock price data in production	Misleading to users. Only real Amazon data shown.
Seed Data	Fake historical data in database for development only	For testing charts and UI without burning API calls.
Hosting Cost	Entirely free stack	No budget. Every service must have a free tier forever.
Development	Local VS Code, not GitHub Codespaces	Developer preference. All env vars managed locally.
Training Data	Synthetic data for ML models	Cannot collect enough real data. Documented in README.

Practical Implications

- With 100 RapidAPI calls/month, you can track a MAXIMUM of 3 products simultaneously in production
- During development, use seed data from the database — do NOT call the real API while building
- Only call the real API when testing the scraper specifically
- Seed data (fake historical prices) exists in database/seed.sql for UI development
- The ML models are trained on synthetic data — this must be documented honestly in the README
- SerpAPI integration is planned but not built yet — the system is designed to add it later
- No Flipkart, Croma, or other platforms until SerpAPI or another free source is added

What We Are NOT Building (Yet)

- Multi-platform price comparison (only Amazon for now)
- Flipkart or Croma scraping
- Real-time WebSocket price updates (not needed at once-per-day scraping)

- Mobile app
- User-uploaded product CSV import
- Social features (sharing deals)
- Browser extension

These exclusions are not failures — they are scope decisions. Mentioning them in interviews as 'future work' actually shows good engineering judgment.

SECTION 3 — Final Tech Stack

Complete Tech Stack With Reasons

Layer	Technology	Hosting	Why This Choice
Frontend	React	Vercel (free)	Component-based UI, industry standard, Recharts for charts
API Gateway	Express.js (Node 18)	Render (free)	Non-blocking I/O, fast, JavaScript consistency
Scraper	Python 3.11	Render background worker	Best async libraries, asyncio for parallel calls
ML Service	Python + Flask	Render (free)	Scikit-learn, Prophet, native Python ML ecosystem
Notifications	Python	Render background worker	Gmail SMTP, Redis pub/sub listener
Database	TiDB Cloud Serverless	TiDB free tier (5GB)	MySQL-compatible, always on, no sleep, free forever
Cache + Queue	Redis (Upstash)	Upstash free tier	Job queue, pub/sub, caching — one tool for three jobs
Price API	RapidAPI (Amazon)	Free (100 calls/month)	Only free option for Amazon India product data
CI/CD	GitHub Actions	Free (public repo)	Auto-deploy on push, lint + test before deploy
Containers	Docker + Docker Hub	Free tier	Consistent environments, easy Render deployment
Frontend Host	Vercel	Free forever	Auto-deploy from GitHub, instant CDN
Backend Host	Render	Free tier	Auto-deploy, background workers, environment vars
Logging	Logtail	Free (1GB/month)	Structured log search across all services
Uptime	UptimeRobot	Free	Keeps Render awake, alerts on downtime
ML Training	Google Colab	Free	Train models without local GPU, save as .pkl
Version Control	GitHub	Free (public)	Required for GitHub Actions CI/CD

Skills This Stack Covers From Your Resume

Resume Skill	Where It Appears in PriceRadar
Python	Scraper, ML service, Notifications service
C++	Not in this project — keep for DSA interviews
MERN Stack	React (frontend) + Node/Express (backend) + MySQL-compatible DB
RESTful APIs	Express API Gateway with full REST endpoints
AWS (EC2, RDS)	Optional — can swap Render for EC2, TiDB for RDS if needed
Docker	All four backend services containerised
Jenkins CI/CD	Replaced by GitHub Actions — same concept, mention both
Linux/Bash	Dockerfile, GitHub Actions YAML, shell commands
MySQL	TiDB is MySQL-compatible — identical SQL syntax
MongoDB	Not in this project — keep for other projects
TensorFlow	Not used — Scikit-learn + Prophet sufficient for these models
NumPy/Pandas	Used in ML training and feature engineering
Scikit-Learn	Random Forest (deal scorer) + Isolation Forest (fake detector)

SECTION 4 — System Architecture

Architecture Overview

PriceRadar follows a microservices-inspired architecture. Each service has one job, communicates through defined interfaces, and fails independently without bringing down the rest of the system.

Services and Their Single Responsibilities

Service	Language	Job
Frontend	React/JS	User interface — search, dashboard, charts, alerts
API Gateway	Node/Express	Single entry point for all frontend requests
Scraper Orchestrator	Python	Manages job queue, picks up scrape jobs, calls aggregator
Price Aggregator	Python	Calls Amazon API, normalises data, stores in DB
ML Service	Python/Flask	Runs 3 models, returns deal score + prediction + anomaly
Notification Svc	Python	Listens for price events, checks targets, sends emails

Communication Between Services

From	To	Method	What
Frontend	API Gateway	HTTP REST	All user actions
API Gateway	TiDB	SQL over TCP	Read/write permanent data
API Gateway	Redis	Redis protocol	Cache reads, rate limit check
API Gateway	ML Service	HTTP REST	Fetch ML predictions
Scraper	Redis	lpush/brpop	Job queue management
Scraper	Amazon RapidAPI	HTTP REST	Fetch current price
Scraper	TiDB	SQL	Store price history
Scraper	Redis pub/sub	PUBLISH	Announce price update
ML Service	TiDB	SQL	Load price history for models
ML Service	Redis	SET with TTL	Cache prediction results
Notif. Svc	Redis pub/sub	SUBSCRIBE	Listen for price updates
Notif. Svc	TiDB	SQL	Check targets, write alerts
Notif. Svc	Gmail SMTP	SMTP over TLS	Send email to user

System Design Patterns Implemented

Pattern	Where	Why
API Gateway	Express.js	Single entry, auth + rate limit in one place
Job Queue	Redis List (brpop)	Async scraping, retry on failure, no data loss
Pub/Sub Events	Redis pub/sub	Decoupled notifications — scraper doesn't know about email
Circuit Breaker	Price Aggregator	If RapidAPI fails, stop hammering, serve cached data
Token Bucket Rate Limiting	API Gateway	Protect free API quotas from abuse
Three-Level Cache	Browser+Redis+TiDB	Minimise API calls, maximise response speed
Read/Write Split	DB connection pools	Separate pools for reads vs writes — scales to replicas
Retry + Dead Letter	Scraper Orchestrator	Failed jobs retry 3x then go to dead letter queue
Structured Logging	All services	JSON logs with consistent fields, searchable in Logtail
Health Endpoint	API Gateway	UptimeRobot pings /health every 5 min to keep Render awake

SECTION 5 — Database Design

TiDB Schema — All Tables

TiDB Cloud Serverless is MySQL-compatible. All standard MySQL syntax works. The database is named 'priceradar'. Six tables cover all data needs.

Table: users

Everyone who registers

Column	Type / Constraint	Purpose
id	INT AUTO_INCREMENT PK	Unique user identifier
email	VARCHAR(255) UNIQUE	Login email
password_hash	VARCHAR(255)	bcrypt hash — never plain text
email_notifications	BOOLEAN DEFAULT TRUE	User preference for email alerts
created_at	TIMESTAMP	Registration time

Table: products

Every product ever searched or tracked

Column	Type / Constraint	Purpose
id	INT AUTO_INCREMENT PK	Unique product identifier
name	VARCHAR(500)	Full product name from Amazon
image_url	TEXT	Amazon product image URL
category	VARCHAR(100)	Electronics / Appliances etc.
asin	VARCHAR(20)	Amazon Standard Identification Number
amazon_url	TEXT	Direct Amazon product link
current_price	DECIMAL(10,2)	Latest known price — updated on each scrape
created_at	TIMESTAMP	When first added to PriceRadar

Table: price_history

CORE TABLE — every price point ever recorded

Column	Type / Constraint	Purpose
id	INT AUTO_INCREMENT PK	Row identifier
product_id	INT FK → products.id	Which product
price	DECIMAL(10,2)	Actual selling price at scrape time

Column	Type / Constraint	Purpose
mrp	DECIMAL(10,2)	Maximum Retail Price shown
discount_percent	DECIMAL(5,2)	Calculated discount percentage
in_stock	BOOLEAN	Was it available at scrape time
scraped_at	TIMESTAMP	Exact time of this price check

Table: tracked_products

Links users to products they watch

Column	Type / Constraint	Purpose
id	INT AUTO_INCREMENT PK	Row identifier
user_id	INT FK → users.id	Which user
product_id	INT FK → products.id	Which product
target_price	DECIMAL(10,2)	Alert when price drops below this
is_active	BOOLEAN DEFAULT TRUE	FALSE when user stops tracking
created_at	TIMESTAMP	When tracking started
UNIQUE	(user_id, product_id)	User can only track same product once

Table: alerts

Every notification ever triggered

Column	Type / Constraint	Purpose
id	INT AUTO_INCREMENT PK	Row identifier
user_id	INT FK → users.id	Who to notify
product_id	INT FK → products.id	Which product triggered alert
message	TEXT	Human-readable alert message
current_price	DECIMAL(10,2)	Price that triggered the alert
is_read	BOOLEAN DEFAULT FALSE	Has user seen this alert
created_at	TIMESTAMP	When alert was triggered

Table: platform_metadata

Health tracking for price API sources

Column	Type / Constraint	Purpose
id	INT AUTO_INCREMENT PK	Row identifier
platform_name	VARCHAR(50) UNIQUE	amazon / flipkart / etc.

Column	Type / Constraint	Purpose
last_successful_scrape	TIMESTAMP	Last time API worked correctly
is_active	BOOLEAN	Should we call this platform now
failure_count	INT DEFAULT 0	Consecutive failures — used by circuit breaker

Indexes

Index	Table	Column(s)	Why
idx_price_history_product_id	price_history	product_id	Most common lookup — get history for one product
idx_price_history_scraped_at	price_history	(product_id, scraped_at)	Time-range queries for chart data
idx_tracked_products_user_id	tracked_products	user_id	Load user's dashboard
idx_tracked_products_product_id	tracked_products	product_id	Find all users tracking a product
idx_alerts_user_id	alerts	user_id	Load user's notifications
idx_alerts_unread	alerts	(user_id, is_read)	Fast unread count query

Service 1: API Gateway (Express.js)

The single entry point for all frontend requests. No frontend component ever talks directly to TiDB or Redis. Everything goes through here.

Middleware Stack (runs in this order on every request)

- CORS middleware — allow requests from Vercel frontend domain
- Body parser — convert JSON request body text into JavaScript object
- Rate limiter — check Redis token bucket for this user's IP
- Auth middleware — verify JWT token on protected routes only
- Route handler — actual business logic
- Error handler — catch all thrown errors, return clean JSON response

Key Design Decisions

- Connection pooling: 3 write connections, 7 read connections to TiDB
- JWT tokens expire after 24 hours — user must re-login
- Rate limit: 100 requests per hour per user using token bucket algorithm
- All errors return consistent format: { error: string, code: number }
- Health endpoint GET /health — used by UptimeRobot to keep Render awake

Service 2: Scraper Orchestrator + Price Aggregator

Two tightly related Python components. The Orchestrator manages the job queue. The Aggregator does the actual price fetching.

Orchestrator — Job Queue Logic

- Runs as infinite loop using brpop (blocking pop) on Redis list
- brpop waits up to 5 seconds for a job — efficient, no busy-waiting
- Picks job from pending_scrapes → moves to processing → calls Aggregator
- On success: deletes from processing, schedules next scrape for 24 hours later
- On failure: retry_count < 3 → retry_queue. retry_count = 3 → dead_letter_queue
- On startup: recovery process moves any stuck processing jobs back to pending

Aggregator — Price Fetching Logic

- Calls RapidAPI Amazon endpoint with product ASIN
- Circuit breaker wraps every external API call
- Normalises response to standard format: { price, mrp, discount_percent, in_stock }
- Stores new row in price_history table in TiDB
- Updates current_price in products table
- Invalidates Redis cache key for this product

- Publishes price_updated event to Redis pub/sub channel

Circuit Breaker States

State	Condition to Enter	Behaviour	Exit Condition
CLOSED (normal)	Default state	All requests go through	Stay closed unless failures accumulate
OPEN (failing)	5 failures in 2 minutes	Reject all requests immediately, serve cached data	After 10 minutes → enter HALF_OPEN
HALF_OPEN (testing)	10 minute timeout elapsed	Allow exactly one test request	Success → CLOSED. Failure → OPEN again

Service 3: Notification Service

Completely decoupled from the scraper. Never called directly — wakes up only when it receives a Redis pub/sub event.

- Subscribes to price_updated Redis channel on startup
- On event: loads all users tracking that product from TiDB
- For each user: checks if new price is below their target_price
- If below target: inserts row in alerts table, sends email via Gmail SMTP
- Email contains: current price, target, deal score from ML, direct Amazon link
- If ML results not yet in Redis cache, skips ML data and sends basic alert

Service 4: ML Service (Flask API)

A lightweight Flask server that loads three pre-trained models on startup and serves predictions via HTTP. Does NOT retrain in production.

- Models loaded from saved_models/*.pkl files at startup — instant loading
- POST /predict: takes product_id, loads price history from TiDB, returns all 3 predictions
- Results cached in Redis with 24-hour TTL (matches scrape frequency)
- If price history has fewer than 7 data points, returns null (not enough data yet)

SECTION 7 — ML Pipeline

Three ML Models

■ All models are trained on SYNTHETIC data generated in Google Colab. This must be documented clearly in the README. It is acceptable for a portfolio project but must not be misrepresented as trained on real Amazon data.

Model 1 — Price Drop Predictor

Attribute	Detail
Library	Facebook Prophet (pip install prophet)
Type	Time series forecasting
Input	Last 30-90 days of price history for a product as { date, price } pairs
Output	Predicted price for next 7 days and 14 days, with confidence range
Minimum data needed	At least 7 data points (7 days of scraping)
Training	Fits fresh on each prediction — Prophet is fast enough, no pickle needed
Shown to user as	'Price predicted to drop to Rs.X in 2 weeks'
Chart display	Dotted line extending the price history chart into the future

Model 2 — Deal Scorer

Attribute	Detail
Library	Scikit-learn Random Forest Regressor
Type	Supervised regression
Input features	discount_percent, historical_avg_price, current_price, days_since_last_drop, pct_above_all_time_low, is_sale_season, price_volatility, category_encoded
Output	Deal score 1-10 (float, rounded to nearest integer)
Training	10,000 synthetic price snapshots with ground truth scores
Ground truth logic	If price stayed low or dropped further after = high score. If price bounced back = low score.
Saved as	saved_models/deal_scorer.pkl
Shown to user as	Coloured badge: 8/10 'Genuinely rare price' or 3/10 'Happens regularly'

Model 3 — Fake Discount Detector

Attribute	Detail
Library	Scikit-learn Isolation Forest
Type	Unsupervised anomaly detection
Input features	mrp_change_7_days, mrp_change_30_days, days_to_nearest_sale_season, discount_jump, mrp_stability_score
Output	Anomaly score: -1 to 1. Above 0.1 = suspicious. Above 0.3 = strong warning.
What it detects	MRP artificially inflated just before a sale to make discount look bigger than it is
Real-world problem	Amazon India sellers inflate MRP before Diwali/Big Billion Days sales — this catches it
Saved as	saved_models/fake_detector.pkl
Shown to user as	Warning banner: 'Original price may have been inflated 3 weeks ago'

Training Workflow (Google Colab)

- Step 1: Run training/generate_data.py — creates synthetic price histories as CSV
- Step 2: Run training/train_deal_scorer.py — trains Random Forest, saves deal_scorer.pkl
- Step 3: Run training/train_fake_detector.py — trains Isolation Forest, saves fake_detector.pkl
- Step 4: Download both .pkl files from Colab
- Step 5: Copy .pkl files into ml/saved_models/ folder
- Step 6: Commit to GitHub — binary files are fine to commit here
- Prophet does NOT need a saved model — it fits fresh on each request (fast enough)

Feature Engineering

The file ml/utils/feature_engineering.py converts raw price history rows from TiDB into the feature vectors the models expect. This runs on every /predict call before passing data to the models.

SECTION 8 — DevOps & Deployment

Docker

Each backend service has its own Dockerfile. Docker Compose ties them together for local development. Render uses the Dockerfiles for production deployment.

Services and Their Dockerfiles

Service	Base Image	Exposed Port	Start Command
backend (Express)	node:18-alpine	5000	node src/server.js
scraper (Python)	python:3.11-slim	none	python main.py
ml (Flask)	python:3.11-slim	8000	python main.py
notifications (Python)	python:3.11-slim	none	python main.py

GitHub Actions CI/CD Pipeline

The pipeline runs automatically on every push to the main branch. If any step fails, deployment is blocked. The old version stays live.

Pipeline Stages

- Stage 1 — Test: Checkout code, install dependencies, run ESLint (JS), Flake8 (Python), pytest
- Stage 2 — Build: Build Docker images for all 4 services, push to Docker Hub tagged with commit SHA
- Stage 3 — Deploy: Trigger Render deploy webhook (backend auto-deploys), Vercel auto-deploys frontend from GitHub push

Render Hosting Setup

Service	Render Type	Free Tier Limit	Keep-Alive Method
API Gateway	Web Service	Sleeps after 15 min idle	UptimeRobot pings /health every 5 min
Scraper	Background Worker	Always on (no sleep)	N/A — background workers don't sleep
ML Service	Web Service	Sleeps after 15 min idle	API Gateway calls it on demand
Notifications	Background Worker	Always on	N/A

Free Tier Summary

Service	Free Tier	Limit	Expires?
TiDB Cloud	Serverless	5GB storage, always on	Never

Service	Free Tier	Limit	Expires?
Upstash Redis	Regional	10,000 commands/day	Never
Render	Web + Workers	Limited hours/month	Never (with limits)
Vercel	Hobby	Unlimited deployments	Never
RapidAPI	Basic	100 calls/month	Never
GitHub Actions	Public repo	2000 min/month	Never
Docker Hub	Free	1 private repo, unlimited public	Never
Logtail	Free	1GB logs/month	Never
UptimeRobot	Free	50 monitors, 5-min interval	Never
Google Colab	Free	GPU access, session limits	Never

Complete Folder Structure

Every file listed below has a specific purpose. Files marked (KEY) are the most important ones — understand these first.

```
priceradar/
|
|-- .github/workflows/deploy.yml [KEY] CI/CD pipeline
|-- database/
| |-- schema.sql [KEY] All CREATE TABLE statements
| |-- indexes.sql All CREATE INDEX statements
| |-- seed.sql Fake data for development only
|
|-- frontend/ React app – deployed to Vercel
| |-- public/index.html Root HTML
| |-- src/
| |-- index.js Mounts React app
| |-- App.js [KEY] Routing setup
| |-- pages/
| | |-- Landing.jsx Home + search bar
| | |-- Login.jsx / Register.jsx Auth forms
| | |-- Dashboard.jsx [KEY] Tracked products grid
| | |-- Product.jsx [KEY] Chart + ML insights + alert setup
| | |-- Alerts.jsx Notification history
| | |-- Analytics.jsx Savings summary
| |-- components/
| | |-- PriceChart.jsx [KEY] Recharts price history graph
| | |-- PlatformTable.jsx Price comparison table
| | |-- MLInsights.jsx Deal score + prediction + warning
| | |-- AlertBell.jsx Notification badge
| |-- hooks/
| | |-- useAuth.js JWT token management
| | |-- useProduct.js React Query product data fetching
| |-- api/
| | |-- client.js [KEY] Axios instance with auth header
| | |-- auth.js / products.js / alerts.js API call functions
| |-- context/AuthContext.js Global login state
|
|-- backend/ Express.js – deployed to Render
| |-- src/
| |-- server.js [KEY] App entry point
| |-- routes/ URL → handler mapping
| |-- controllers/ [KEY] Business logic per endpoint
| |-- middleware/
| | |-- auth.middleware.js [KEY] JWT verification
| | |-- rateLimit.middleware.js Token bucket via Redis
| | |-- errorHandler.middleware.js Catch-all error formatter
| |-- services/
| | |-- cache.service.js Redis get/set helpers
| | |-- queue.service.js Push jobs to Redis
```

```
| | |-- ml.service.js Fetch ML predictions
| |-- db/
| | |-- connection.js [KEY] TiDB connection pools
| | |-- queries/ SQL query functions per table
| |-- utils/
| |-- jwt.js generateToken, verifyToken
| |-- password.js hashPassword, comparePassword
| |-- logger.js Structured JSON logger
|
|-- scraper/ Python – Render background worker
| |-- main.py [KEY] Entry point
| |-- orchestrator/
| | |-- worker.py [KEY] Infinite loop, job queue
| | |-- scheduler.py Schedule next scrape
| | |-- recovery.py Recover stuck jobs on startup
| |-- aggregator/
| | |-- aggregator.py [KEY] Calls Amazon API
| | |-- normalizer.py Standardise API response format
| | |-- circuit_breaker.py [KEY] Circuit breaker class
| |-- platforms/
| | |-- rapidapi.py [KEY] RapidAPI Amazon integration
| | |-- mock.py Fake data (development only)
| |-- db/connection.py + queries.py TiDB for Python
| |-- cache/redis_client.py Redis for Python
|
|-- ml/ Python Flask – deployed to Render
| |-- main.py [KEY] Flask server entry point
| |-- api/routes.py POST /predict endpoint
| |-- models/
| | |-- price_predictor.py Prophet model
| | |-- deal_scorer.py Random Forest model
| | |-- fake_detector.py Isolation Forest model
| |-- training/ Run in Google Colab only
| | |-- generate_data.py Create synthetic training data
| | |-- train_deal_scorer.py Train + save deal_scorer.pkl
| | |-- train_fake_detector.py Train + save fake_detector.pkl
| |-- saved_models/ [KEY] Committed .pkl files
| | |-- deal_scorer.pkl
| | |-- fake_detector.pkl
| |-- utils/feature_engineering.py [KEY] Raw data -> model features
|
|-- notifications/ Python – Render background worker
| |-- main.py [KEY] Subscribes to Redis pub/sub
| |-- listener/pubsub_listener.py Receives price_updated events
| |-- handlers/price_alert_handler.py [KEY] Check targets, trigger email
| |-- email/
| |-- sender.py [KEY] Gmail SMTP
| |-- templates/price_alert.html Email HTML template
|
|-- docker-compose.yml [KEY] Local development setup
|-- README.md [KEY] Project documentation
```

SECTION 10 — Development Phases & Current Status

Development Phases

Phase	Name	Status	What Gets Built
0	Setup	COMPLETE	GitHub repo, accounts, folder structure, .env files
1	Database	COMPLETE	TiDB schema, indexes, seed data — all 6 tables live
2	Backend Skeleton	NEXT	Express server, DB connection, Redis connection, all routes as stubs
3	Authentication	PENDING	Register, login, JWT middleware, password hashing
4	Product Endpoints	PENDING	Search (with cache), track product, price history endpoint
5	Price Aggregator	PENDING	RapidAPI integration, normaliser, circuit breaker
6	Scraper Orchestrator	PENDING	Job queue, worker loop, retry logic, recovery
7	ML Service	PENDING	Colab training, Flask API, 3 models, feature engineering
8	Notification Service	PENDING	Pub/sub listener, alert handler, Gmail SMTP
9	Frontend	PENDING	All React pages, charts, ML display, auth flow
10	Docker	PENDING	Dockerfiles, docker-compose, test all services together
11	CI/CD + Deployment	PENDING	GitHub Actions, Render deploy, Vercel deploy, UptimeRobot
12	Polish + README	PENDING	Error handling, screenshots, architecture diagram, README

Phase 2 — Next Steps (Backend Skeleton)

The next phase builds the Express.js backend from zero. Follow this exact order — do not skip ahead.

Step	Action	Test By
2.1	Create package.json, install express/cors/dotenv/mysql2/ioredis/bcryptjs/jsonwebtoken	npm install completes without errors
2.2	Write server.js — basic Express app listening on port 5000	Hit localhost:5000 in browser, see something
2.3	Add GET /health route returning { status: ok }	Hit localhost:5000/health, see JSON response
2.4	Write db/connection.js — TiDB read + write pools	Run SELECT 1, see result in console

Step	Action	Test By
2.5	Write cache/redis_client.js — Upstash connection	redis.set('test','hi'), redis.get('test'), see 'hi'
2.6	Write utils/logger.js — JSON structured logger	See JSON log line on server startup
2.7	Add CORS, body-parser middleware to server.js	POST with JSON body works without errors
2.8	Create all route files with stub handlers returning 501	Thunder Client: all endpoints return 501 message
2.9	Add error handler middleware last in server.js	Throw test error, see clean JSON error response

SECTION 11 — Environment Variables Reference

All Environment Variables by Service

■ NEVER commit .env files to Git. All .env files are in .gitignore. Only .env.example files (with empty values) are committed.

File: backend/.env

Variable	Example Value	Notes
TIDB_HOST	gateway01.ap-southeast-1.prod.aws.tidbcloud.com	TiDB cluster host
TIDB_PORT	4000	TiDB port (always 4000)
TIDB_USER	2FmGxxxxxx.root	TiDB username from console
TIDB_PASSWORD	your_generated_password	TiDB password — generated once
TIDB_DATABASE	priceradar	Database name
REDIS_URL	rediss://default:pass@xxx.upstash.io:6379	Upstash Redis connection string
JWT_SECRET	64+ character random string	Sign/verify JWT tokens — keep secret
ML_SERVICE_URL	http://localhost:8000	ML Flask service URL (localhost in dev)
PORT	5000	Express server port
LOGTAIL_TOKEN	your_logtail_token	Structured log shipping

File: scraper/.env

Variable	Example Value	Notes
RAPIDAPI_KEY	your_rapidapi_key	From RapidAPI dashboard
RAPIDAPI_HOST	the-api-host.p.rapidapi.com	Shown in API docs on RapidAPI

File: ml/.env

Variable	Example Value	Notes
PORT	8000	Flask server port

File: notifications/.env

Variable	Example Value	Notes
GMAIL_USER	priceradar.alerts@gmail.com	Dedicated Gmail account
GMAIL_APP_PASSWORD	abcdefghijklmnop	16-char app password, no spaces

File: frontend/.env

Variable	Example Value	Notes
REACT_APP_API_URL	http://localhost:5000	Backend URL (change to Render URL after deploy)

SECTION 12 — API Endpoints Reference

All API Endpoints

Auth Endpoints

Method	Path	Auth?	Request Body	Response
POST	/auth/register	No	{ email, password }	{ token, user }
POST	/auth/login	No	{ email, password }	{ token, user }

Product Endpoints

Method	Path	Auth?	Request Body	Response
GET	/products/search?query=X	Yes	None	Array of products with current price
POST	/products/track	Yes	{ product_id, target_price }	{ success: true }
GET	/products/tracked	Yes	None	User's tracked products with ML data
GET	/products/:id	Yes	None	Single product details
GET	/products/:id/history	Yes	None	Price history array for chart
DELETE	/products/:id/untrack	Yes	None	{ success: true }

Alert Endpoints

Method	Path	Auth?	Request Body	Response
GET	/alerts	Yes	None	All alerts for user (newest first)
GET	/alerts/unread	Yes	None	{ count: N }
PATCH	/alerts/:id/read	Yes	None	{ success: true }
PATCH	/alerts/read-all	Yes	None	{ updated: N }

Analytics Endpoints

Method	Path	Auth?	Request Body	Response
GET	/analytics/summary	Yes	None	Total savings, best deal found, total alerts
GET	/analytics/history	Yes	None	Monthly savings breakdown

System Endpoints

Method	Path	Auth?	Request Body	Response
GET	/health	No	None	{ status: ok, timestamp }

ML Service Endpoint (internal — not exposed to frontend directly)

Method	Path	Request Body	Response
POST	/predict	{ product_id: 123 }	{ deal_score: 7, predicted_price_7d: 44000, predicted_price_14d: 42000, fake_discount: false, anomaly_score: -0.05 }

What to Say in Interviews

These are the answers to the most likely interview questions about this project. Practise these until they feel natural. Every answer is backed by real implementation.

Q: Tell me about your most interesting project.

A: PriceRadar is a multi-service price tracking system for Amazon India. It runs three ML models in production — a Prophet time series predictor, a Random Forest deal scorer, and an Isolation Forest fake discount detector. The interesting engineering challenge was building it entirely on free infrastructure while implementing real system design patterns: job queues, pub/sub, circuit breakers, and three-level caching. It's live at [your URL] if you want to see it.

Q: How does your caching work?

A: Three levels. Browser level: React Query caches API responses for 5 minutes — repeat page visits don't hit the backend at all. Application level: Redis caches search results for 1 hour and ML predictions for 24 hours. Database level: TiDB has built-in query caching for repeated identical reads. A popular product tracked by 50 users hits the Amazon API once per day — all 50 users get cached data.

Q: Explain your notification architecture.

A: It's event-driven using Redis pub/sub. The scraper never knows notifications exist — it just publishes a `price_updated` event after storing new data. The notification service is independently subscribed to that channel. This means if the notification service crashes, scraping continues unaffected. Adding a new downstream consumer — say an analytics service — means just subscribing it to the channel. Zero changes to the scraper.

Q: What is a circuit breaker and why did you use one?

A: A circuit breaker wraps external API calls and tracks failure rate. If the Amazon RapidAPI fails 5 times in 2 minutes, the circuit opens — all further requests are rejected immediately without trying the API. After 10 minutes it allows one test request. If that succeeds, it closes. Without this, rate limit errors would cascade through the system. With it, we serve cached prices gracefully and the user sees a staleness warning instead of an error.

Q: How would you scale this to 10 million products?

A: The scraper workers are stateless — they pull from a Redis queue and process. Scaling means adding more worker containers. No code changes needed. For the database, partition `price_history` by `product_id` range — each shard handles a subset of products. For the ML service, pre-compute predictions on a schedule and store results, rather than computing on-demand. The API gateway already separates read and write connections, so adding read replicas to TiDB is a config change, not a code change.

Q: Why did you choose Python for the scraper and not Node.js?

A: Three reasons. First, `asyncio` gives Python true async concurrency for I/O-bound tasks like API calls. Second, the ML pipeline is Python — keeping the scraper in the same language means shared utility code

and simpler deployment. Third, libraries like requests, aiohttp, and APScheduler are mature and well-documented for this exact use case.

Q: What is your fake discount detector and how does it work?

A: It's an Isolation Forest model — unsupervised anomaly detection. It learns what normal MRP behaviour looks like from historical data. If an MRP suddenly increases by 30% in the week before a sale, then drops back with a 'discount', the short isolation path flags it as anomalous. This catches a real problem in Indian e-commerce that existing tools like CamelCamelCamel don't address. It's my most unique differentiator.

How to Use This Document With AI Assistants

When starting a new conversation with Claude, ChatGPT, or Copilot about this project, paste the following context block at the start of your prompt. This gives the AI everything it needs to give accurate, project-specific answers without you having to re-explain the architecture every time.

Master Context Block — Paste This at the Start of Every New AI Conversation

```
I am building a project called PriceRadar.

PROJECT SUMMARY:
PriceRadar is a multi-service full-stack web application that tracks Amazon India product prices, runs ML models to analyse deals, and emails users when their target price is hit.
It is a portfolio project built entirely on free infrastructure.

TECH STACK:
- Frontend: React, hosted on Vercel
- API Gateway: Express.js (Node 18), hosted on Render
- Scraper: Python 3.11, runs as Render background worker
- ML Service: Python Flask, hosted on Render
- Notifications: Python, runs as Render background worker
- Database: TiDB Cloud Serverless (MySQL-compatible), free tier
- Cache + Queue: Upstash Redis, free tier
- Price API: RapidAPI Amazon (Real-Time Amazon Data by OpenWebNinja), 100 calls/month free
- CI/CD: GitHub Actions on public repo
- Containers: Docker + Docker Hub

HARD CONSTRAINTS:
- Only 100 RapidAPI calls/month – maximum 3 products tracked simultaneously
- Amazon India only (no Flipkart/SerpAPI yet)
- Scrape once per day only to conserve quota
- No mock/fake price data in production
- Seed data (fake historical prices) exists in database only for development/testing
- Entirely free stack – no paid services
- Development in local VS Code

CURRENT STATUS:
- Phase 0 (Setup): COMPLETE – accounts created, folder structure built, .env files done
- Phase 1 (Database): COMPLETE – TiDB schema, indexes, seed data all live
- Phase 2 (Backend Skeleton): IN PROGRESS – starting now

DATABASE TABLES: users, products, price_history, tracked_products, alerts, platform_metadata

FOLDER STRUCTURE:
priceradar/
```

```
backend/ (Express.js API Gateway)
scraper/ (Python scraper + price aggregator)
ml/ (Python Flask ML service)
notifications/ (Python notification service)
frontend/ (React)
database/ (SQL files)

ML MODELS:
1. Prophet – price drop prediction (7 and 14 days)
2. Random Forest – deal score 1-10
3. Isolation Forest – fake discount detection
All trained on synthetic data in Google Colab, saved as .pkl files

SYSTEM DESIGN PATTERNS IMPLEMENTED:
API Gateway, Job Queue (Redis), Pub/Sub (Redis), Circuit Breaker,
Token Bucket Rate Limiting, 3-Level Caching, Read/Write DB Split,
Retry + Dead Letter Queue, Structured Logging

Now help me with: [YOUR SPECIFIC QUESTION]
```

Specific Prompt Templates

When asking for code for a specific file:

```
Using the project context above, write the code for [filename]. It should [what it does]. Use [specific libraries]. Connect to TiDB using the TIDB_* env vars. Follow the existing pattern where [relevant pattern].
```

When debugging:

```
Using the project context above, I am getting this error in [service name]: [paste error]. The relevant file is [filename]. Here is the code: [paste code]. What is wrong and how do I fix it?
```

When asking about architecture decisions:

```
Using the project context above, should I [decision]? Consider our constraints: 100 API calls/month, free hosting only, once-per-day scraping. What are the tradeoffs?
```

When starting a new phase:

```
Using the project context above, I am starting Phase [N] which is [phase name]. Walk me through step by step what to build first, in what order, and how to test each step before moving on.
```

What NOT to Ask AI (Common Mistakes)

- Do NOT ask AI to write the entire project in one prompt — you won't understand it
- Do NOT use AI-generated code you cannot explain line by line — interviewers will know
- Do NOT ask AI to 'make it work' without understanding why it was broken
- Do NOT skip the test step after each piece — bugs compound across phases
- DO use AI to explain concepts you don't understand before writing the code yourself

- DO use Copilot inline to complete code you already know how to write
- DO ask AI to review your code after you've written it